Concurrent and Distributed Systems

The 80/20 Revision Guide

Part IB — Cambridge CST — Paper 5 (Dr M. Kleppmann)

How this guide is organised

The course is **two halves** — *Concurrent Systems* (one machine, shared memory) and *Distributed Systems* (many machines, message passing) — and the two Paper-5 questions span both. Across **2018–2025 (16 questions)**:

Topic	Appearances	Share
Synchronization (semaphores, monitors, locks)	7	44%
Fault tolerance & system models	6	38%
Message passing / RPC / broadcast	5	31%
Concurrency primitives & atomicity	5	31%
Distributed consistency models	4	25%
Logical clocks; deadlock; replication/SMR	3 each	19%
Leader election; transactions; time sync; resource policies	2 each	12%

80/20 verdict. On the concurrent side, **synchronization** (monitors/semaphores) and **concurrency primitives** dominate; deadlock and transactions are the next tier. On the distributed side, **fault models**, **message passing/RPC**, **logical clocks**, and **replication/consistency** are the spine. The defining idea of the distributed half: **unbounded delay + partial failure** make it fundamentally different from concurrency — you can't tell "crashed" from "slow".

PART A — Concurrent Systems

1 Concurrency primitives & atomicity (Tier 1 — 31%)

Exam-critical

Threads share memory and are **interleaved** (and preempted) arbitrarily ⇒ **race conditions**. A **critical section** needs **mutual exclusion**. **Atomicity:** an operation appears indivisible. Software-only mutual exclusion is hard (Lamport's **bakery algorithm** does it without special instructions); in practice we use **hardware atomics**:

test-and-set, **compare-and-swap (CAS)**, **load-linked/store-conditional (LL/SC)**.

A correct lock needs mutual exclusion, progress (no deadlock), and bounded waiting.

2 Synchronization primitives (Tier 1 — 44%, the biggest topic)

Exam-critical

[title=Semaphores] A **semaphore** holds a count with atomic wait (P: decrement, block if < 0) and signal (V: increment, wake one). **Binary** (mutex) vs **counting**. Used for mutual exclusion *and* condition synchronization, e.g. **producer–consumer** with a bounded buffer: a mutex + an "empty slots" semaphore + a "full slots" semaphore.

Exam-critical

[title=Monitors and condition variables] A **monitor** bundles shared state with procedures under *automatic* mutual exclusion (only one thread active inside). **Condition variables** let a thread `wait` (release the monitor and block) and `signal` (wake a waiter) on a named condition. Maintain a **monitor invariant** that holds whenever the lock is free.

Exam trap

Hoare vs Mesa signalling is the classic exam point. Under **Hoare** semantics the signalled thread runs *immediately* (so the condition still holds). Under **Mesa** semantics (Java, pthreads) the signaller keeps running and the waiter is merely made runnable — the condition may be false again by the time it runs. **Therefore always re-test in a `while` loop, never an `if`:** `while (!cond) wait();`. Java's `notify`/`notifyAll`, pthreads, OpenMP and Cilk are all Mesa-style.

3 Deadlock & liveness (Tier 2 — 19%)

Exam-critical

Deadlock requires all **four Coffman conditions** simultaneously: *mutual exclusion*, *hold-and-wait*, *no preemption*, *circular wait*. Strategies:

- **Prevention** — deny one condition (e.g. a global *lock ordering* kills circular wait).
- **Avoidance** — the Banker's algorithm: only grant a request if a safe sequence still exists.
- **Detection & recovery** — find a cycle in the **resource-allocation graph**, then abort/roll back.

Also know **livelock** (threads active but make no progress) and **priority inversion** (low-priority holder blocks high-priority thread; fix with *priority inheritance*).

4 Transactions & concurrency control (Tier 2 — 12%)

Exam-critical

ACID: Atomicity, Consistency, Isolation, Durability. **Serializability** is the gold standard for isolation: a concurrent execution is correct if equivalent to *some* serial order — test by checking the **precedence (conflict) graph** is acyclic. Achieve it by:

- **Two-phase locking (2PL)** — acquire all locks (growing phase) before releasing any (shrinking); *strict* 2PL holds write locks to commit (avoids cascading aborts).
- **Timestamp ordering** — order transactions by timestamp, abort out-of-order accesses.

- **Optimistic concurrency control (OCC)** — execute on a private copy, then *validate* for conflicts at commit; good under low contention.

Worth knowing

Recovery & lock-free. **Write-ahead logging (WAL)** + **checkpoints** give durability/atomicity: log before applying, replay (redo) committed and roll back (undo) uncommitted on crash. **Lock-free** data structures use CAS retry loops (progress without locks — no deadlock, but watch the ABA problem); **HTM/STM** (hardware/software transactional memory) offer optimistic atomic blocks.

PART B — Distributed Systems

5 System models & fault tolerance (Tier 1 — 38%)

Exam-critical

The two things that make distribution hard: **unbounded message delay** and **partial failure** (some nodes fail while others continue; you cannot distinguish a crashed node from a slow one or a lost message).

- **Timing models:** *synchronous* (bounded delay/clock skew), *asynchronous* (no bounds), *partially synchronous* (bounds hold eventually) — the realistic one.
- **Fault models:** *crash-stop*, *crash-recovery*, and *Byzantine* (arbitrary/malicious) — increasing difficulty.

Two Generals problem: agreement is *impossible* over an unreliable channel — no finite protocol guarantees both sides know they agree. Motivates "best-effort + retries + idempotence".

6 RPC & communication (Tier 1 — 31%)

Exam-critical

Remote procedure call makes a network call look like a local one: an **IDL** defines the interface, stubs **marshal**/unmarshal arguments. But the abstraction *leaks* because of partial failure — a lost reply is indistinguishable from a crashed server. Hence delivery semantics:

- **at-least-once** — retry on no reply; needs *idempotent* operations;
- **at-most-once** — never re-execute (reply may be lost);
- **exactly-once** — achievable only with de-duplication + persistence.

Worth knowing

Other models: the **actor** model and **CSP**/tuple-spaces pass messages instead of sharing memory (no locks). **Broadcast/multicast** abstractions are built on best-effort messaging + retransmission — see logical clocks for their ordering guarantees.

7 Time & logical clocks (Tier 1/2 — 19%)

Exam-critical

Physical clocks drift; **NTP** synchronises them but never perfectly, so **don't use wall-clock time to order events**. Use **causality**: the **happens-before** relation $a \to b$ (same process order, or send-before-receive, transitively).

- **Lamport clocks:** a single counter; $a \to b \Rightarrow L(a) < L(b)$, but *not* the converse — gives a consistent *total* order, not causality detection.
- **Vector clocks:** one entry per process; $a \to b \iff V(a) < V(b)$ — *characterise* causality, so they detect concurrent events.

Worth knowing

Broadcast ordering (increasing strength): **FIFO** (messages from one sender in order) $\subseteq$ **causal** (respects happens-before) $\subseteq$ **total-order** (all nodes deliver in the *same* order — the key primitive for replication). **Gossip** spreads updates epidemically for scalability.

8 Replication & consistency (Tier 1/2 — 25% + 19%)

Exam-critical

State-machine replication (SMR): run the same deterministic state machine on every replica and feed them the *same* commands via **total-order broadcast** $\Rightarrow$ identical state. **Leader-based replication** funnels writes through a leader. **Quorums:** with N replicas, reads R and writes W overlap iff $R + W > N$.

Consistency models: linearizability (strongest — every operation appears to take effect atomically at a point between its call and return; reads see the latest write) vs **eventual consistency** (replicas converge if updates stop). **Read-after-write** sits between.

Exam trap

CAP theorem (state it carefully): during a *network partition* you must choose **consistency** (linearizability) *or* **availability** — not both. It says nothing when there's no partition. Don't confuse **linearizability** (a *recency* guarantee on single objects) with **serializability** (a *transaction-isolation* guarantee). **Spanner** uses **TrueTime** (bounded-uncertainty GPS/atomic clocks) to get external consistency by *waiting out* the uncertainty.

Worth knowing

ABD algorithm: linearizable read/write register on crash-prone replicas via majority quorums (read-then-write-back). **CRDTs** (conflict-free replicated data types): operations designed to *commute*, so replicas converge without coordination — the basis of **collaborative text editing**. **Two-phase commit (2PC):** atomic commit across nodes (prepare $\to$ commit/abort) — but it *blocks* if the coordinator crashes (not fault-tolerant; consensus fixes this).

9 Consensus (Tier 2 — 12%)

Exam-critical

Consensus: all nodes agree on one value (agreement, validity, termination). **FLP impossibility:** *no deterministic* consensus algorithm guarantees termination in an *asynchronous* system with even *one* crash fault. We escape it in practice via **partial synchrony** (timeouts) or randomisation. **Raft:** a leader-based algorithm — *leader election* (randomised election timeouts + terms) then *log replication* (leader appends, replicates to a majority, commits) — chosen for understandability over Paxos. 2PC is *not* consensus (it blocks); consensus tolerates coordinator failure.

Exam checklist

Exam-critical

1. **Primitives:** race conditions, mutual exclusion, atomicity; test-and-set / CAS / LL-SC; bakery algorithm.
2. **Synchronization:** semaphores (producer–consumer), monitors + condition variables; **Hoare vs Mesa** signalling ⇒ `while` not `if`.
3. **Deadlock:** four Coffman conditions; prevention (lock ordering) / avoidance (Banker's) / detection (RAG cycle); livelock; priority inversion → inheritance.
4. **Transactions:** ACID; serializability via acyclic precedence graph; 2PL (strict), timestamp ordering, OCC; WAL + checkpoints; lock-free CAS.
5. **Distributed models:** unbounded delay + partial failure; sync/async/partially-sync; crash-stop/crash-recovery/Byzantine; two generals.
6. **RPC:** marshalling, IDL, partial failure ⇒ at-least-once (idempotent) / at-most-once / exactly-once.
7. **Clocks:** happens-before; Lamport (total order, not causality) vs vector (characterises causality); FIFO $\subseteq$ causal $\subseteq$ total-order broadcast.
8. **Replication/consistency:** SMR via total-order broadcast; quorums $R + W > N$; linearizability vs eventual; **CAP** (only during a partition); CRDTs commute; 2PC blocks.
9. **Consensus:** agreement/validity/termination; **FLP** (async + 1 crash); escape via partial synchrony; **Raft** (election + log replication).